

Enacting the Question of Self

Provenance, a hypergraph, and a differentiable route that records computation as data

Brent Hartshorn
brenthartshorn@proton.me

Abstract

A companion to *Staging the Question of Self* [1], which built a self-contained, self-modifying PDF that poses an evolving sequence of questions about selfhood without claiming to answer them. Here we extend that apparatus from posing questions to *enacting computations beside them*, under a single discipline: computation is recorded as data, never as an answer. We give a formal account of four developments and diagram their mechanisms. (i) *Provenance*: a stage extends a parent by its DOI and ingests the parent’s relations, a monotone operation that lets the inquiry look fully back along a published lineage. (ii) A *hypergraph*: relations are generalised to hyperedges over arbitrary node sets, with binary relations as the two-node case. (iii) *Fuzzy and precise*: each node pairs a suggestive Greek glyph with a precise description, labelling the interpretive slack rather than hiding it. (iv) A *differentiable route*: following the `@dslai` pattern of [6], a node’s specification compiles into a small trainable network, making the hypergraph partly differentiable; we give its forward map, loss, and gradient step explicitly. We argue that learning and action do not move the system closer to demonstrating selfhood; they *distribute the authorship* into the loss, the architecture, the data, and the action-mapping, which makes the data-not-answer discipline load-bearing rather than decorative. A worked example — an autoencoder compiled from the node named *self*, trained to reconstruct its own input — is the sharpest specimen of the temptation the discipline refuses.

1 Introduction: from staging to enacting

The first paper [1] took a deliberately modest stance toward a grand question. It built an instrument that *stages* “what is a self?” — makes the asking concrete, evolving, documented, and reproducible — and was candid that the asking is authored and the apparatus answers nothing. The published series (stages 0–3) [2, 3, 4, 5] bears this out: each stage’s abstract ends “it poses them, and makes no claim to resolve them.”

This paper reports what happens when the apparatus is allowed to *do* things beside the questions: relate many terms at once, attach precise descriptions, trigger computations, and compile a node into a trainable module. The risk in every one of these is identical and worth naming at the outset. A system that not only asks “what is the self?” but *computes* and *learns* looks far more like an emerging mind than one that merely asks. That appearance is exactly what must not be cashed as a claim. The contribution is therefore as much methodological as technical: a set of extensions, each formalised below, paired with one rule — record computation as data, never as an answer — that keeps the added power from quietly becoming an overclaim.

2 Provenance: extending a document by DOI

Write a document state at stage n as a labelled hypergraph $G_n = (V_n, E_n)$ (defined in Section 3). Each stage names a *parent* by its archival DOI and reconstructs the parent’s relations on load. Let

$\rho(G) \subseteq E$ denote the inherited relational core (the binary relations and the operator definitions they use). Stage n is formed as

$$G_n = \text{ingest}(G_{\text{parent}}) \oplus \Delta_n, \quad \rho(G_n) \supseteq \rho(G_{\text{parent}}), \quad (1)$$

where `ingest` reads the parent and Δ_n is the oracle-supplied increment (new nodes, hyperedges, operator code). The containment $\rho(G_n) \supseteq \rho(G_{\text{parent}})$ is the formal content of “looking fully back”: the inherited core grows monotonically, $\rho(G_0) \subseteq \rho(G_1) \subseteq \dots$, even as new questions are posed on top of it. Because every stage is a self-contained, self-extracting artifact (Section 10), the DOI of stage n is a complete and reproducible specification of how stage $n+1$ was reached. Figure 1 shows the loop that realises this.

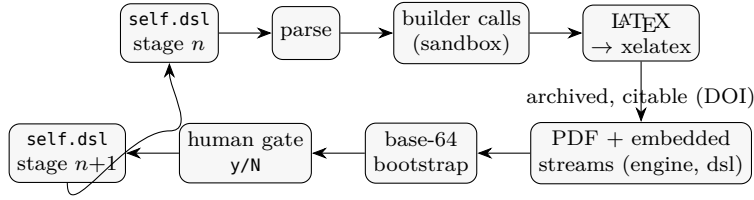


Figure 1: The provenance loop. A stage’s DSL is parsed into sandboxed builder calls, rendered to a PDF that embeds its own engine and source, and reduced to a base-64 bootstrap; a human gate admits the successor, which re-enters as the next stage’s source. Each archived PDF is a citable parent.

3 A hypergraph of the inquiry

Let V be a finite set of *nodes*. Each node $v \in V$ is a triple $v = (g_v, w_v, p_v)$: a glyph g_v , a word w_v , and a precise description p_v (Section 4); a node may additionally carry a module specification m_v (Section 6). A *hyperedge* is a triple

$$e = (S_e, q_e, \alpha_e), \quad S_e \subseteq V, \quad |S_e| \geq 1, \quad (2)$$

where q_e is a question and $\alpha_e \in \mathcal{A} \cup \{\perp\}$ an optional action (Section 5). A binary relation $a \odot b$ is the special case $S_e = \{a, b\}$ in which the operator $\odot$ fixes the question via a template $Q_\odot$, so $q_e = Q_\odot(a, b)$. Retaining binary relations as two-node hyperedges is what lets the parent’s structure be inherited without translation (Section 2). Figure 2 draws a stage: seven nodes, a selection of binary relations, and the hyperedge $\{\Sigma, \Lambda, \mu\}$ as a region enclosing three nodes at once.

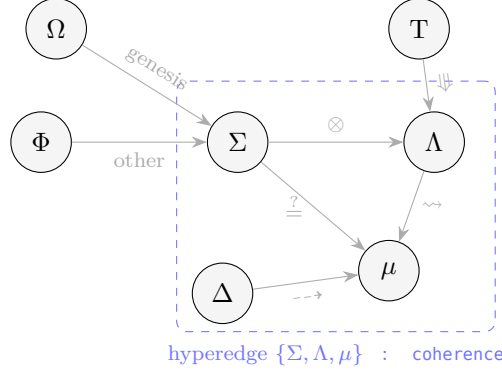


Figure 2: A hypergraph stage. Arrows are inherited binary relations (two-node hyperedges) carrying the parent’s operators ($\otimes$ reached-through, $\rightsquigarrow$ haunting, $\Rightarrow$ inscription, $--\rightarrow$ rupture, $\stackrel{?}{=}$ identity-in-question). The dashed region is a genuine hyperedge over $\{\Sigma, \Lambda, \mu\}$ with the action **coherence**.

4 Fuzzy and precise: labelling the slack

A recurring hazard is that an evocative symbol is mistaken for content. The first paper noted that a Greek letter used as a variable carries connotation to a human reader but no extra meaning to the program. Rather than choose between the suggestive and the precise, each node carries both: a glyph g_v (e.g. Σ), a word w_v (*self*), and a precise description p_v (“the locus that poses and is altered by the question”). The glyph is for the eye, the description is for the argument, and pairing them localises the interpretive slack at g_v instead of letting typography smuggle it in unannounced. The same split appears in DSLs that pair machine-precise rules with natural-language instructions meant for an interpreter to read [12].

5 Actions: questions that trigger computation

An action is a map

$$\alpha : (S, G) \mapsto d \in \mathcal{D}, \quad (3)$$

from a node set and the document state to a datum d (a number, vector, or record). It runs only behind an explicit gate; when it runs, the engine records the *pair* $\langle q_e, d \rangle$ and never the substitution $q_e \mapsto d$. The honesty invariant is typed: q_e lives in the slot of questions, d in the slot of data, and there is no coercion $\mathcal{D} \rightarrow \text{Answers}(q_e)$. Concretely, the hyperedge $\{\Sigma, \Lambda, \mu\}$ with action **coherence** — a deterministic overlap measure on the three precise descriptions — records $d = \{\text{coherence} : 0.042, \text{shared} : [\text{“the”}]\}$: the only word the descriptions share is “the.” The procedure returned a number; the question stands beside it, unanswered. A second hyperedge naming no action ($\alpha = \perp$) simply remains a question. The contrast is the design.

6 The @dslai route: nodes that compile to trainable modules

The strongest extension makes the hypergraph partly differentiable. Following the @dslai pattern [6], a node may carry a specification $m_v = (d_0, d_1, d_2, \sigma)$ — written `module Sigma : 3 -> 8 -> 3 : sigmoid`

— that compiles to a trainable map $f_\theta : \mathbb{R}^{d_0} \rightarrow \mathbb{R}^{d_2}$,

$$h = \sigma(W_1 x + b_1), \quad W_1 \in \mathbb{R}^{d_1 \times d_0}, \quad b_1 \in \mathbb{R}^{d_1}, \quad (4)$$

$$o = \sigma(W_2 h + b_2), \quad W_2 \in \mathbb{R}^{d_2 \times d_1}, \quad b_2 \in \mathbb{R}^{d_2}, \quad (5)$$

with $\sigma(z) = (1 + e^{-z})^{-1}$ elementwise and parameters $\theta = \{W_1, b_1, W_2, b_2\}$ (Figure 3). For a task (X, Y) with $X \in \mathbb{R}^{N \times d_0}$, $Y \in \mathbb{R}^{N \times d_2}$, training minimises the mean squared error

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \|f_\theta(x_i) - y_i\|_2^2, \quad \theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}, \quad (6)$$

by gradient descent. Writing $O = f_\theta(X)$ and H for the hidden activations, with $\odot$ the elementwise product, the gradients are

$$\delta_2 = \frac{2}{N} (O - Y) \odot O \odot (1 - O), \quad \nabla_{W_2} = \delta_2^\top H, \quad \nabla_{b_2} = \sum_i (\delta_2)_i, \quad (7)$$

$$\delta_1 = (\delta_2 W_2) \odot H \odot (1 - H), \quad \nabla_{W_1} = \delta_1^\top X, \quad \nabla_{b_1} = \sum_i (\delta_1)_i. \quad (8)$$

A subset of nodes thus carries parameters θ that gradient descent can move: the hypergraph is partly differentiable. (The implementation is a dependency-light NumPy perceptron so the engine stays self-contained inside the PDF; the pattern is identical with a tensor library [8].)

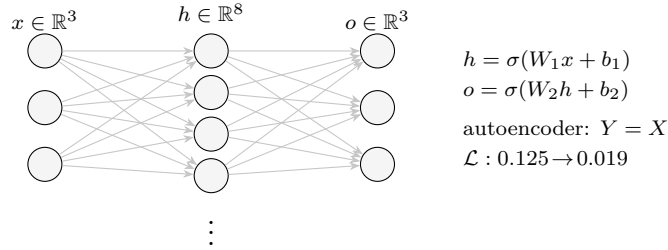


Figure 3: The node Σ compiled from $3 \rightarrow 8 \rightarrow 3$: **sigmoid**. For the worked example the target is the input itself ($Y = X$): an autoencoder. Training drops the reconstruction error from 0.125 to 0.019 — recorded as data, not as an answer.

The worked example is chosen to make the temptation acute. The hyperedge $\{\Sigma\}$ asks “can the self reconstruct itself from its own compression?” and its action **train** fits Σ ’s module — an autoencoder, whose purpose is to reconstruct its input through a bottleneck — to exactly that. It works: $\mathcal{L}$ falls from 0.125 to 0.019. It would be effortless to write that “the self learned to reconstruct itself” and let the reader hear “the self came to know itself.” The engine refuses the sentence: the recorded datum reads, verbatim, *an optimization result over a defined toy objective; not the self learning what it is*. Figure 4 shows the rendered stage; the success is real, the resonance is supplied by the reader, and the two are kept in separate slots.

Λ language — *the finite medium any self must pass through*
 μ meaning — *what a reader supplies, or finds, in the passage*
 Φ other — *the one whose regard first gives the self to itself*
 T time — *the direction in which the self cannot return*
 Δ difference — *the gap that lets one thing be told from another*

Hypergraph (questions, and the data actions returned)

- $\{\Sigma\}$? Can the self reconstruct itself from its own compression? [train]
 $\hookrightarrow$ recorded data (not an answer): {'node': 'Sigma', 'arch': '3->8->3', 'task': 'autoencode', 'mse_start': 0.1252, 'mse_final': 0.0186, 'is': 'an optimization result over a defined toy objective; not the self learning what it is'}
- $\{\Sigma, \Lambda, \mu\}$? What of the self survives being carried at once into language and meaning? [coherence]
 $\hookrightarrow$ recorded data (not an answer): {'coherence': 0.042, 'shared': ['the']}

Questions this stage poses

- Does Ω give rise to Σ — and what precedes Ω ?
- On what does Σ entailing Σ rest?
- When Σ is carried by Λ , what survives and what is lost?
- Is $\partial\Sigma$ the same as μ , or is μ supplied by the reader?
- If Σ does not reduce to Ω , where is the remainder?
- Does Φ give rise to Σ — and what precedes Φ ?
- Does Φ haunt Σ — present as an absence?
- What residue remains because T leaves a trace upon Λ ?
- Does $\partial\Lambda$ haunt μ — present as an absence?
- What collapses when Δ shatters the structure of μ ?
- When ∂T is carried by Δ , what survives and what is lost?
- Can the self reconstruct itself from its own compression?
- What of the self survives being carried at once into language and meaning?

Lineage and provenance

This model extends its parent self-evolving inquiry, doi:10.5281/zenodo.20707850, ingesting its relations so the series can look fully back [1, 2, 3, 4]. Actions record data beside questions; the document poses and enacts, and makes no claim to resolve what a self is.

Appendix: self-extraction

This PDF carries its engine (introspecter2.py) and DSL (self.dsl) as embedded streams. Requires `pip install pypdf numpy` and a LaTeX install. To produce the next stage, save and run:

```
import base64
exec(base64.b64decode(
    "aW1wb3J0IHNScywgZ2xvYiwgb3MKc3lzLnBhdGguaW5zZXJ0KDasICl0Ii"
    "kKZnJvbSBweXBkZiBpbXBvcnQUGRmUmVhZGVyCnBkZiA9IG1heChnbG91"
```

Figure 4: A rendered stage. The hyperedge $\{\Sigma\}$ triggered `train`; the autoencoder’s optimization result is recorded *as data*, explicitly not as an answer to the question above it. The hyperedge $\{\Sigma, \Lambda, \mu\}$ likewise records a near-empty coherence measurement.

7 What changes, honestly, when authorship is distributed

The first paper located the system’s limit in one observation: because the document can be rewritten to ask anything, the apparent depth of any stage is supplied by whoever authored it. Actions and learning do not lift this limit; they relocate it. With a hand-written oracle the authorship was one

person at a keyboard, visible. Once a question triggers a computation, authorship spreads into the procedure α ; once a node compiles to a *trained* module, it spreads further — into the loss $\mathcal{L}$, the architecture (d_0, d_1, d_2, σ) , the data (X, Y) , and the map from question to action. None of these looks like a choice on its own, yet together they fix everything the system will “find.” Distributed authorship is harder to see than a keyboard, which is precisely why the typed separation of q from d stops being a courtesy and becomes the mechanism that keeps the work honest. A trained network answering “what survives translation?” returns a number meaning “what $\mathcal{L}$ rewarded”; an untrained one returns noise; neither is the self speaking. The discipline is to render the number as a number.

8 Related work

Domain-specific languages that describe or compile neural networks are by now well established. DeepDSL specifies a network as a composition of mathematical functions and compiles it for efficient execution [8]; SEMKIS-DSL specifies dataset and recognition-skill *requirements* for an NN-based system [9]; and Hinkel et al. integrate neuronal networks into robot control through a DSL in a model-driven pipeline [10]. Our `@dslai` route (Section 6) shares their move from a declarative specification to a runnable network, but inverts the purpose: the network is compiled *into an inquiry*, and its output is deliberately withheld from the role of an answer. Two further lines are relevant. DSLs that pair machine-precise constructs with natural-language instructions for an AI interpreter — `ai.txt` being a recent instance [12] — mirror our fuzzy/precise node pairing (Section 4), though there the natural language is meant to be *acted on* whereas ours is meant to be *argued with*. And DSLs that foreground *provenance* and social context for ML datasets [11] parallel our DOI-based provenance (Section 2), with the difference that our lineage is the object of study, not metadata about it. To our knowledge, the distinguishing feature here is not any single mechanism but their combination under one constraint: a differentiable, self-evolving document that records computation as data and refuses to mistake it for understanding.

9 Safety: gated execution, carried forward

The apparatus executes code: oracle blocks that extend the engine, and actions that train modules. From the predecessor we carry one invariant — generated or evolved code is inert text until a human confirms it — and apply it uniformly: code blocks re-activate only on explicit `y`, and actions run only when gated (Figure 1). We also note a deliberate omission. The architecture we borrow the `@dslai` pattern from [6] includes a mode that executes a network’s predicted code automatically during training and a routine that spawns a copy of the running program. We adopt the declarative-module idea and decline the ungated auto-execution and self-spawning; at this scale they are harmless, but they are the one pattern not worth normalising. This is appropriate for a single trusted user running a personal instrument and is not intended for untrusted input.

10 Reproducibility and the published lineage

Every stage remains a single self-contained PDF that carries its engine and its source as embedded file streams; a short base-64 appendix extracts and runs them to produce the successor (Figure 1). The predecessor series is archived with permanent identifiers [1, 2, 3, 4, 5], and each new stage names its parent’s DOI, so the lineage is reconstructable end to end from public artifacts. The

engine is maintained in a public repository (Appendix A) and the stage source is reproduced in Appendix B.

11 Conclusion

We have extended a question-staging instrument into one that also enacts computations, relates terms in a hypergraph, pairs suggestive symbols with precise descriptions, and compiles a node into a trainable module — and we have given the formal account and diagrams that make each mechanism explicit. The single rule that computation is recorded as data, never as an answer, does more work here than before, because learning and action distribute the authorship into places that no longer look authored. The autoencoder named *self*, succeeding at reconstructing its own input while the engine declines to call that self-knowledge, is the compact emblem of the project: it does the most suggestive thing it can, and says plainly that it has not thereby answered anything.

Acknowledgments

The implementation and this draft were developed in dialogue with an AI assistant; the honest framing is endorsed by the author.

References

- [1] B. Hartshorn, *Staging the Question of Self: A self-contained, self-modifying document that mixes poetry, philosophy, and code*. doi:10.5281/zenodo.20709754
- [2] B. Hartshorn, *Origin, Self, Language, and Meaning: A Self-Evolving Inquiry* (self_0). doi:10.5281/zenodo.20707685
- [3] B. Hartshorn, *...and Other* (self_1). doi:10.5281/zenodo.20707727
- [4] B. Hartshorn, *...and Time* (self_2). doi:10.5281/zenodo.20707775
- [5] B. Hartshorn, *...and Difference* (self_3). doi:10.5281/zenodo.20707850
- [6] B. Hartshorn, *Self-Contained Multi-AI Architecture Definition via DSL*. viXra:2507.0074
- [7] B. Hartshorn, *Emergent Self-Modification and Meta-Programming in Dynamic Systems*. viXra:2507.0036
- [8] T. Zhao and X. Huang, *Design and implementation of DeepDSL: A DSL for deep learning*. Computer Languages, Systems & Structures **54** (2018) 39–70.
- [9] B. Jahić, N. Guelfi, and B. Ries, *SEMKIS-DSL: A Domain-Specific Language to Support Requirements Engineering of Datasets and Neural Network Recognition*. Information **14**(4):213 (2023). doi:10.3390/info14040213
- [10] G. Hinkel, H. Groenda, L. Vannucci, O. Denninger, N. Cauli, and S. Ulbrich, *A Domain-Specific Language (DSL) for Integrating Neuronal Networks in Robot Control*. Proc. 2015 Joint MORSE/VAO Workshop, ACM, pp. 9–15. doi:10.1145/2802059.2802060
- [11] J. Giner-Miguel, A. Gómez, and J. Cabot, *A Domain-Specific Language for Describing Machine Learning Datasets*. Proc. MODELS '22, ACM (2022). arXiv:2207.02848

- [12] Y. Li, W. Song, B. Zhu, D. Gong, Y. Liu, G. Deng, C. Chen, L. Ma, J. Sun, T. Walsh, and J. Xue, *ai.txt: A Domain-Specific Language for Guiding AI Interactions with the Internet*. arXiv:2505.07834 (2025).

A Implementation availability

The complete engine — `introspector2.py`, the code carried inside each stage’s PDF as an embedded stream — is maintained in the public repository <https://github.com/brentharts/introspector> (file `introspector2.py`). It is omitted here for brevity; given the engine, the stage source in Appendix B reproduces a stage.

B Stage source (`self.dsl`)

```
/* introspector2 - stage 5 ; the @dslai route : a node compiles to a trainable module */
nu : 5 ;
parent : 10.5281/zenodo.20707850 ;

Omega : origin      : "that from which the self is said to come" ;
Sigma : self        : "the locus that poses and is altered by the question" ;
Lambda : language   : "the finite medium any self must pass through" ;
mu     : meaning     : "what a reader supplies, or finds, in the passage" ;
Phi    : other       : "the one whose regard first gives the self to itself" ;
Tau    : time        : "the direction in which the self cannot return" ;
Delta  : difference  : "the gap that lets one thing be told from another" ;

module Sigma : 3 -> 8 -> 3 : sigmoid ;

Omega -> Sigma ;      /* genesis */
Sigma |= Sigma ;     /* the reflexive claim */
Sigma (x) Lambda ;   /* reached only through language */
@Sigma ?= mu ;       /* is the self's changing the same as meaning? */
Sigma -< Omega ;     /* not merely its origin */
Phi -> Sigma ;       /* the self is given by the other */
Phi ~> Sigma ;       /* the other haunts the self */
Tau *> Lambda ;      /* time writes upon the word */
@Lambda ~> mu ;      /* the shift in language is haunted by vanished meaning */
Delta !> mu ;        /* difference shatters stable meaning */
@Tau (x) Delta ;     /* the passage of time is carried by difference */

{ Sigma } ? "Can the self reconstruct itself from its own compression?" ! train ;
{ Sigma, Lambda, mu } ? "What of the self survives being carried at once into language and meaning?" ! coherence
```